

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



CARÁTULA PARA ENTREGA DE PRÁCTICAS

Profesor:	Dr. Sergio Teodoro Vite
Grupo de Laboratorio:	06
Número de práctica:	06
Título de la práctica:	Texturizado
Fecha de entrega:	13 de octubre de 2026

Número de cuenta	Nombre completo	Grupo de teoría
321137135	Basilio Illescas Andrés	04
424065465	Cruz Macedo Samuel Santiago	04
321258173	Medel Piña Ángel de Jesús	02
321150503	Pérez Olvera Alexis Abraham	04
320224159	Segura Cedeño Luisa María	02

HOJA DE EVIDENCIAS

PRÁCTICA 06: TEXTURIZADO

Lista de números de cuenta exclusivamente:

321137135
424065465
321258173
321150503
320224159

Resumen

En esta práctica se asoció información de textura a modelos tridimensionales para representar objetos reales a partir de primitivas: un plano con una hoja recortada sobre fondo transparente, un cubo convertido en caja de cartón, un cilindro convertido en barril y una esfera convertida en la Tierra, además de una variación de cada uno (reja de malla ciclónica, edificio de ladrillo con tinaco y balón de fútbol). Las figuras, el despliegue de coordenadas UV y los materiales se configuraron íntegramente en la interfaz interactiva de Blender 5.2 utilizando fotografías y texturas libres (ambientCG, Solar System Scope, Wikimedia Commons) con dimensiones en potencias de dos, asignando en cada caso las coordenadas UV adecuadas a la primitiva: planas, por cara, cilíndricas y equirrectangulares; el barril recibió además su forma abombada con edición proporcional. Los modelos se exportaron a FBX y se cargaron en OpenGL 3.3 con Assimp, y el mapa de entorno del escenario se sustituyó por el cubemap *Footballfield* de Humus. Como resultado, las ocho figuras se visualizan correctamente en OpenGL con su transparencia, y en el camino resolvimos tres fallas del cargador: la omisión de las transformaciones de los nodos del FBX, la ausencia de normales en las mallas suaves y la escritura de profundidad de los fragmentos transparentes.

Palabras clave: Texturizado, coordenadas UV, atlas de texturas, canal alfa, mapeo cilíndrico y esférico, cubemap, Blender, OpenGL, Assimp.

Abstract

In this laboratory session, texture information was associated with three-dimensional models in order to represent real objects starting from primitives: a plane carrying a leaf cut out over a transparent background, a cube turned into a cardboard box, a cylinder turned into a barrel and a sphere turned into the Earth, plus one variation of each (a chain-link fence, a brick building with a water tank and a football). The models, UV coordinate mapping, and materials were configured entirely within the interactive interface of Blender 5.2 using freely licensed photographs and resources (ambientCG, Solar System Scope, Wikimedia Commons) into textures whose dimensions are powers of two, and each primitive received the UV coordinates that suit it: planar, per face, cylindrical and equirectangular; the barrel was additionally given its bulging shape with proportional editing. The models were exported to FBX and loaded in OpenGL 3.3 through Assimp, and the scene environment map was replaced with Humus' *Footballfield* cubemap. As a result, all eight figures are displayed correctly in OpenGL, including their transparency, and along the way we solved three loader issues: FBX node transforms being ignored, smooth meshes arriving without normals, and transparent fragments writing to the depth buffer.

Keywords: Texturing, UV coordinates, texture atlas, alpha channel, cylindrical and spherical mapping, cubemap, Blender, OpenGL, Assimp.

1. Introducción

En las prácticas anteriores las figuras de la escena se distinguían únicamente por su geometría: el color de cada cara era uniforme, o bien lo daba una textura que ya venía incluida en el modelo. Representar con esa estrategia un objeto real, como una caja de cartón o un barril, exigiría modelar cada veta, cada cinta y cada remache con polígonos. El texturizado resuelve el problema de otra manera: la geometría se queda simple y el detalle se toma de una imagen que se «pega» sobre la superficie. La idea se remonta a la tesis de Catmull [8], que propuso subdividir cada parche de superficie hasta el tamaño de un píxel y tomar su color de una imagen, y fue generalizada por Blinn y Newell [6], quienes además introdujeron el uso de una imagen del entorno para simular reflejos. Desde entonces el mapeo de texturas es una de las etapas básicas del cauce de renderizado [10, 1].

Coordenadas de textura

Para asociar una imagen a una malla, a cada vértice se le asigna, además de su posición, un par de coordenadas $(u, v) \in [0, 1]^2$ que indica qué punto de la imagen le corresponde. Durante la rasterización estas coordenadas se interpolan en el interior de cada triángulo y el *fragment shader* consulta la imagen con ellas [14]:

$$\mathbf{c}(x, y) = T\big(u(x, y),\ v(x, y)\big), \tag{1}$$

donde T es la textura y (u, v) las coordenadas interpoladas en el fragmento (x, y). Cuando una coordenada sale del intervalo $[0, 1]$, el modo de repetición (`GL_REPEAT`) hace que la imagen se repita, lo que permite cubrir superficies grandes con una sola imagen continua en sus bordes [13].

El problema central del texturizado es, por lo tanto, encontrar la función que lleva cada punto de la superficie a un punto del plano (u, v), lo que en Blender se llama *despliegue* o *UV unwrapping* [5]. Para las cuatro primitivas de esta práctica existe una proyección natural:

- **Plano:** la proyección es directa, $u = (x - x_{\text{mín}})/a$ y $v = (z - z_{\text{mín}})/h$, con a y h el ancho y el alto del plano.
- **Cubo:** cada cara se proyecta sobre el plano de sus dos ejes y se coloca en una región distinta de la imagen. A la imagen que reúne varias regiones independientes se le llama *atlas* de texturas.
- **Cilindro:** el costado se «desenrolla» usando el ángulo alrededor del eje y la altura, $u = \theta/2\pi$ con $\theta = \operatorname{atan2}(y, x)$, y $v = z/h$; las tapas se proyectan en planta.
- **Esfera:** se usa la proyección equirrectangular, en la que u es la longitud y v la latitud:

  $$u = \frac{\lambda}{2\pi}, \qquad v = \frac{\varphi + \pi/2}{\pi}, \qquad \mathbf{d} = (\cos\varphi\cos\lambda,\ \cos\varphi\sin\lambda,\ \sin\varphi). \tag{2}$$

  Es la misma proyección con la que se distribuyen los mapas de planetas, por lo que una esfera UV de Blender acepta esas imágenes sin deformarlas.

En el cilindro y en la esfera aparece una costura donde u salta de 1 a 0; si un triángulo cruza esa línea y no se corrige, la interpolación recorre toda la imagen en sentido inverso dentro de ese triángulo y aparece una franja con la textura completa comprimida.

Canal alfa y transparencia

Una imagen RGBA guarda, además del color, un valor de opacidad α por píxel. Porter y Duff [15] formalizaron cómo combinar un color que se dibuja (fuente) con el que ya está en pantalla (destino); el operador *over* que usa el código del laboratorio es

$$\mathbf{c} = \alpha_s\,\mathbf{c}_s + (1 - \alpha_s)\,\mathbf{c}_d, \tag{3}$$

que en OpenGL corresponde a `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. La mezcla, sin embargo, no impide que un fragmento totalmente transparente escriba su profundidad en el *z-buffer*, y si eso ocurre, lo que se dibuje después detrás de él queda oculto. Por eso los objetos transparentes se dibujan al final o se descartan sus fragmentos con α pequeña [1].

Dimensiones en potencias de dos

Las imágenes cuyas dimensiones son potencias de dos (512, 1024, 2048, ...) permiten construir la pirámide de *mipmaps* reduciendo cada nivel exactamente a la mitad, técnica propuesta por Williams [18] para evitar el parpadeo de las texturas lejanas. Aunque OpenGL 3.3 ya acepta otras dimensiones, decidimos que todas las texturas de esta práctica midieran potencias de dos, porque así se conserva la compatibilidad con hardware antiguo y con las rutinas que generan *mipmaps*.

Mapas de entorno

Un mapa de entorno representa todo lo que rodea a la escena como si estuviera infinitamente lejos. Greene [9] propuso guardarlo en las seis caras de un cubo (*cubemap*), cada una con un campo de visión de 90°, de modo que juntas cubren todas las direcciones sin las deformaciones que tiene una sola imagen esférica. Las caras se nombran por el eje hacia el que miran (`posx`, `negx`, `posy`, `negy`, `posz`, `negz`), y es la convención que siguen los mapas de Humus [12] que pide usar el manual.

Desarrollo de la práctica

Siguiendo el manual [17], en esta práctica texturizamos un plano, un cubo, un cilindro y una esfera para representar respectivamente una hoja con fondo transparente, una caja de cartón, un barril y un planeta; los exportamos a FBX y los cargamos en OpenGL con Assimp [4], y cambiamos el mapa de entorno del escenario. Además, el equipo decidió hacer una variación por cada primitiva (una reja, un edificio con su tinaco y un balón), y en todas las texturas combinamos al menos dos imágenes. Las fotografías provienen de bibliotecas con licencias libres: ambientCG [2], Solar System Scope [16], Wikimedia Commons [11, 7] y Humus [12].

El desarrollo se llevó a cabo en Blender 5.2 y Visual Studio sobre Windows 11, realizando el modelado, el desenvolvimiento de coordenadas UV, la asignación de materiales y la exportación de los modelos directamente dentro de la interfaz gráfica e interactiva de Blender. Durante el desarrollo se utilizó además un modelo de lenguaje como apoyo para la depuración del cargador en OpenGL y la revisión de redacción del documento [3].

2. Metodología

2.1. Configuración del entorno de trabajo

En la Tabla 1 se resumen las herramientas empleadas. El proyecto de OpenGL es el `06_Texturing` del repositorio del laboratorio, que ya incluye la clase `Model` (`model.h`) basada en Assimp, un *shader* que dibuja únicamente el color de la textura y un cubo de entorno (`mycube.fbx`).

Elemento	**Especificación / Versión empleada**
Sistema operativo	Windows 11 (64 bits)
Entorno de desarrollo	Microsoft Visual Studio Community (conjunto de herramientas `v145`)
Compilador y estándar	MSVC, `C++20`, configuración `Debug|x64`
API gráfica	OpenGL 3.3 (perfil núcleo)
Bibliotecas	GLFW, GLAD, GLM, Assimp (`assimp-vc143-mtd`) y `stb_image`
Edición de imágenes	GIMP 2.10 (recorte y generación del canal alfa)
Modelado y texturas	Blender 5.2.2 LTS

Tabla 1: Herramientas, bibliotecas y versiones utilizadas.

2.1.1. Creación del proyecto en Visual Studio

Como en las prácticas anteriores, dentro de la carpeta `06_Texturing` se creó la carpeta del proyecto `Proyecto06` con la plantilla *Proyecto vacío* de C++, se agregaron los archivos `06_Texturing.cpp` y `stb_image.cpp`, y se configuraron las propiedades de la Tabla 2 para las configuraciones `Debug|x64` y `Release|x64`. Además de GLFW y GLAD, que indica el manual, fue necesario agregar Assimp, porque `model.h` incluye `<assimp/Importer.hpp>`: sin esa biblioteca el proyecto compila pero no enlaza.

Propiedad	**Valor**
Directorio de salida y de trabajo	`$(ProjectDir)..\..\bin\`
Directorio de inclusión adicionales	`deps\glm`, `deps\glad`, `deps\glfw`, `deps\assimp` e `include` del proyecto
Directorios de bibliotecas adicionales	`deps\glad`, `deps\glfw` y `deps\assimp` (`lib\x64\Debug`)
Dependencias adicionales	`opengl32.lib; glad.lib; glfw3.lib; assimp-vc143-mtd.lib`
Definiciones del preprocesador	`_CRT_SECURE_NO_WARNINGS` (por `stb_image`)

Tabla 2: Propiedades del proyecto `Proyecto06` en Visual Studio.

2.1.2. Procedimiento común en Blender

El modelado de las primitivas, el desenvolvimiento de sus coordenadas UV y la configuración de los materiales se realizaron directamente en la interfaz interactiva de Blender (modo edición, editor de sombreado y editor UV), organizando todos los objetos dentro de la misma escena. A excepción de la textura de la hoja (cuyo canal alfa se preparó en GIMP), las texturas se prepararon a partir de fotografías y mapas libres, guardándose en formato PNG con dimensiones en potencias de dos. La Tabla 3 resume las imágenes empleadas en cada caso.

Textura	**Tamaño**	**Imágenes que combina**
Hoja	1024^2 RGBA	Foto de una hoja [11] + canal alfa generado en GIMP
Caja	2048^2	Cartón kraft [7] + pestaña y cinta
Barril	2048^2	Duelas `Planks005` + aros `Metal021` + tapas `Planks012` [2]
Planeta	2048×1024	Mapa diurno de la Tierra [16]
Reja	1024^2 RGBA	`Metal011` [2] + malla de rombos calculada como canal alfa
Edificio	2048^2	`Bricks005` + `Concrete006` + `Planks012` [2] + cielo del cubemap [12]
Tinaco	1024^2	`Plastic006` [2] + estrías y rosca
Balón	2048×1024	`Leather012` [2] + patrón del icosaedro truncado

Tabla 3: Texturas de la práctica, su tamaño y las imágenes que se combinaron en cada una.

2.2. Actividad 1: Textura en un plano

2.2.1. Selección de la imagen

Para que el plano mostrara algo más que una imagen rectangular decidimos usar una textura con transparencia: una hoja recortada sobre un fondo transparente, que en OpenGL deja ver el escenario alrededor de su contorno. La fotografía que elegimos [11] (Figura 1) mide 719×717 píxeles y, al revisar el encabezado del PNG, encontramos que su tipo de color es 2 (RGB), es decir, no trae canal alfa: había que generarlo nosotros.



Figura 1: Fotografía original de la hoja (719×717, RGB, sin canal alfa).

2.2.2. Generación del canal alfa en GIMP

Para dotar a la fotografía de transparencia y aislar con precisión la silueta de la hoja, se procesó la imagen original mediante la aplicación de edición gráfica GIMP a través de los siguientes pasos:

1. **Adición del canal alfa a la capa.** Dado que la imagen original se encontraba en formato RGB estándar (tipo de color 2, sin soporte de opacidad), al abrirla en GIMP se añadió explícitamente un canal alfa a la capa activa desde el menú *Capa* → *Transparencia* → *Añadir canal alfa*, habilitando la estructura de 4 canales (RGBA).

2. **Selección y delimitación de la silueta.** Se empleó la *Herramienta de selección difusa* (varita mágica) junto con la *Herramienta de rutas* para perfilar con fidelidad el contorno irregular de la hoja de arce y aislarla del fondo, delimitando y excluyendo la zona inferior donde aparecía el pulgar que sujetaba la hoja en la fotografía original.

3. **Eliminación del fondo.** Se invirtió la selección (*Seleccionar* → *Invertir*) y se suprimió el contenido del fondo mediante la tecla `Supr`, asignando un valor de alfa nulo ($\alpha = 0$) a todos los píxeles externos a la hoja.

4. **Suavizado de bordes (*antialiasing*).** Para evitar cortes abruptos y bordes dentados al momento de renderizar en OpenGL, se aplicó un calado ligero (*Seleccionar* → *Difuminar...* con un radio de 2 a 3 píxeles), generando una transición suave en el perímetro de la hoja.

5. **Recorte y escalado a potencia de dos.** La imagen se recortó a proporción cuadrada y se reescaló a una resolución de 1024×1024 píxeles (*Imagen → Escalar la imagen...*) para garantizar dimensiones en potencias de dos, facilitando el muestreo eficiente por parte del hardware gráfico.

6. **Exportación en formato PNG RGBA.** Finalmente, la textura se exportó como archivo PNG de 32 bits (`hoja_rgba.png`) preservando íntegro el canal de opacidad.

La Figura 2 muestra el resultado obtenido compuesto sobre un fondo magenta de contraste para hacer visible la transparencia: la hoja conserva su forma y textura opaca (33 % de los píxeles), mientras que el área circundante es completamente transparente.



Figura 2: Textura `hoja_rgba.png` (1024×1024, RGBA) compuesta sobre magenta: lo magenta es la parte transparente.

2.2.3. Plano y material

La creación de la geometría, la configuración del material y la asignación de la textura se realizaron directamente dentro de la interfaz de Blender. Se creó una primitiva de plano (*Plane*) de 2 unidades por lado centrada en el origen, cuyas coordenadas UV por defecto ya cubren en su totalidad el espacio $[0, 1]^2$. En el editor de sombreado (*Shader Editor*), se configuró el material conectando un nodo *Image Texture* con la imagen `hoja_rgba.png` a las entradas *Base Color* y *Alpha* del nodo *Principled BSDF*, asignando en los ajustes del material el modo de mezcla (*Blend Mode*) a *Alpha Blend* para que el visor 3D represente la transparencia en tiempo real. Finalmente, el plano se exportó desde Blender a `Blender/plano_hoja.fbx`.

2.3. Actividad 2: Cubo como caja de cartón

2.3.1. Atlas de la caja

Partimos de una fotografía de cartón kraft liso [7], recortada a cuadrado y reducida a 512×512. Con ella se arma un atlas de 2048×2048 dividido en una cuadrícula de 4×4 celdas de 512 píxeles, de las que se usan seis, una por cara (Figura 14). A cada celda se le oscurecen las orillas un 25 % para marcar las aristas de la caja; la tapa y el fondo llevan la línea donde se juntan las pestañas y encima una cinta canela que ocupa el 20 % de la celda, y en las dos caras laterales $\pm X$ la cinta baja un 28 % de la altura, justo donde termina la de la tapa.

2.3.2. Modelado y coordenadas UV por cara

En Blender se creó una primitiva de cubo (*Cube*) de 2 unidades por lado y se colocó sobre el plano en $(3, 0, 1)$. En modo edición, el mapeo de coordenadas UV se configuró manualmente utilizando el editor UV de Blender (*UV Editor*). Cada una de las seis caras del cubo se proyectó y ajustó sobre su celda correspondiente del atlas, cuidando la orientación para que la parte superior de las caras laterales coincida con el eje $+Z$ y las cintas de la tapa continúen de manera uniforme hacia los costados. Se dejó un margen de 4 píxeles respecto a los bordes de cada celda para evitar artefactos en el filtrado bilineal en OpenGL. En el editor de sombreado (*Shader Editor*), se configuró el material conectando la textura del atlas a la entrada *Base Color* del nodo *Principled BSDF*.

2.4. Actividad 3: Cilindro como barril

2.4.1. Atlas del barril

El atlas del barril mide 2048×2048 y combina tres imágenes de ambientCG [2]. La mitad superior es el costado: la textura de tablas `Planks005`, girada 90° para que las tablas queden verticales como duelas y llevada a un tono roble, con cuatro aros recortados de la foto de acero oxidado `Metal021` al 10, 31, 69 y 90 % de la altura. Cada aro tiene un sombreado en coseno que simula su volumen, una sombra sobre la madera y 16 remaches. La mitad inferior contiene las dos tapas: la madera más oscura `Planks012` dentro de un círculo, con un borde de acero del 10 % del radio. Escogimos `Planks005` entre tres candidatas porque sus tablas son largas y continuas; las otras dos son de piso y tienen juntas a tope que en un barril se verían como cortes en las duelas.

2.4.2. Cilindro y coordenadas UV

Se creó una primitiva de cilindro de 32 lados, radio 1 y altura 2.4, posicionada en $(6, 0, 1{,}2)$. En el modo edición de Blender, se marcó una costura vertical (*Mark Seam*) a lo largo de una arista lateral para permitir el desenvolvimiento cilíndrico ordenado sin cruces de textura en la unión. En el editor UV de Blender, la malla rectangular del costado se ajustó sobre la mitad superior del atlas. Para las tapas, se aplicó una proyección planar ortogonal en planta, posicionando cada círculo en su respectiva región de la mitad inferior del atlas (invirtiendo horizontalmente la tapa inferior).

2.4.3. Forma de barril con edición proporcional

Para dar al cilindro la silueta curva característica de un barril, se emplearon las herramientas de modelado de Blender:

1. En Modo Edición, se utilizó la herramienta de cortes en bucle (*Loop Cut*) para subdividir las aristas verticales del costado en 9 cortes, creando anillos horizontales intermedios.
2. Se seleccionó el anillo central de 32 vértices en la altura media ($z = 0$).
3. Se escaló dicho anillo a un factor de 1.18 en los ejes X e Y con la *edición proporcional* (*Proportional Editing*) activada, utilizando caída *Suave* (*Smooth*) y un radio de influencia de 1.2 unidades (media altura del barril).

Con esta configuración, la deformación alcanza su valor máximo en el centro y se desvanece suavemente hacia las tapas. Dado que la interpolación de coordenadas UV se gestiona de forma automática en Blender, las duelas y los aros de la textura acompañan la curvatura del barril sin distorsión.

2.5. Actividad 4: Esfera como planeta

Para el planeta usamos el mapa diurno de la Tierra de Solar System Scope [16], de 2048×1024 píxeles en proyección equirrectangular (Figura 3). La esfera es una *UV Sphere* de 64 segmentos y 32 anillos (1 986 vértices y 2 048 caras), radio 1, en $(-3, 0, 1)$ y con sombreado suave. Como Blender ya asigna a la esfera UV las coordenadas equirrectangulares (longitud en u, latitud en v), el planeta no necesitó ningún despliegue adicional: basta con poner la imagen en el material.

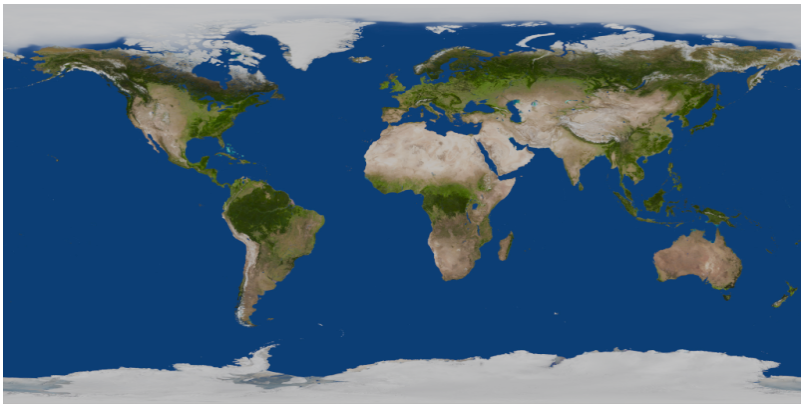


Figura 3: Mapa diurno de la Tierra de Solar System Scope (2048×1024, proyección equirrectangular).

2.6. Actividad 5: Cambio del mapa de entorno

El manual pide sustituir el fondo por imágenes de Humus [12]. Al momento de hacer la práctica el sitio `humus.name` no estaba disponible (el servidor respondía con una página de «sitio no disponible»), así que descargamos el archivo `Footballfield.zip` desde la copia que guarda el Internet Archive. Escogimos este mapa, una cancha de pasto con cielo nublado, porque combina con las variaciones de la práctica, en particular con el balón y la reja. Sus seis caras miden 2048×2048 y se distribuyen con licencia CC BY 3.0. La Figura 4 muestra las seis caras acomodadas en cruz.



Figura 4: Las seis caras del cubemap *Footballfield* de Humus en forma de cruz: `posy` arriba, `negy` abajo y, de izquierda a derecha, `negx`, `posz`, `posx` y `negz`.

El cubo de entorno del laboratorio (`mycube.fbx`) tiene seis materiales que buscan sus imágenes con la ruta relativa `mycube.fbm\posx.jpg` y equivalentes. En lugar de sobrescribir esas imágenes, que usan otras prácticas, creamos la carpeta `bin/models/cubemap_football` con una copia de `mycube.fbx` y su propia carpeta `mycube.fbm` con las seis caras de Humus. Como los dos mapas siguen la misma convención de nombres, cada imagen cae en su cara sin modificar el modelo, y en el código solo cambia la ruta:

```
// Actividad 5: cubemap "Footballfield" de Humus (CC BY 3.0)
cubeenv = new Model("models/cubemap_football/mycube.fbx");
```

Listing 1: Carga del nuevo mapa de entorno en 06_Texturing.cpp.

2.7. Variaciones: una por cada primitiva

Además de las actividades del manual, realizamos una variación de cada primitiva geométrica, colocadas en una segunda fila detrás de las figuras base. Todo el modelado, despliegue de coordenadas UV, preparación de materiales y asignación de texturas de estas variantes se llevó a cabo íntegramente dentro de la interfaz interactiva de Blender.

2.7.1. Plano: reja de malla ciclónica

En Blender se creó un plano vertical de 6×2 unidades ubicado en $(2, 4{,}5, 1)$ para modelar un tramo de cerca perimetral. En el editor de sombreado (*Shader Editor*), se configuró un material con el nodo *Principled BSDF* utilizando una textura RGBA de 1024×1024 con patrón de rombos semitransparente, compuesta con el acabado metálico galvanizado `Metal011` [2]. Se conectó el color a *Base Color* y el mapa de opacidad a *Alpha*, activando el modo *Alpha Blend* para permitir ver a través de los orificios. En el editor UV de Blender, las coordenadas u del plano se escalaron de 0 a 3, logrando que el patrón de malla ciclónica se repita tres veces a lo ancho gracias al modo `GL_REPEAT`. Adicionalmente, se modelaron dentro de Blender dos primitivas de cilindro para incorporar el poste vertical de soporte y el travesaño horizontal superior con material metálico opaco.

2.7.2. Cubo: edificio de ladrillo

Se modeló a partir de una primitiva de cubo escalada a $3\times3\times6$ unidades en modo edición, conformando una estructura de tres plantas. El despliegue de sus coordenadas UV se realizó de forma interactiva en la vista de edición UV de Blender, vinculando sus caras a un atlas de texturas de 2048×2048: la mitad superior se repartió en cuatro cuadrantes para las fachadas (proporción 1:2), integrando el ladrillo `Bricks005` con detalles de concreto `Concrete006` [2] en zócalos, cornisas y molduras de ventanas, las cuales incorporan fragmentos del cielo del cubemap de Humus para simular reflejos en los vidrios. La mitad inferior del atlas se asignó a la azotea y la base, incorporando una puerta de madera `Planks012` en la planta baja y un pretil de concreto en el techo.

2.7.3. Cilindro: tinaco

Se creó una primitiva de cilindro de radio 0.55 y altura 1.1 colocada en la azotea del edificio. En Blender se aplicó un desenrollado UV cilíndrico en el cuerpo lateral y una proyección planar circular para las tapas. En el *Shader Editor* se asignó una textura de 1024×1024 basada en el plástico negro `Plastic006` [2], configurando el material para reproducir las estrías horizontales a lo largo del cuerpo cilíndrico y la rosca concéntrica en la tapa superior. Se aplicó sombreado suave a las caras laterales para garantizar una silueta curva continua.

2.7.4. Esfera: balón de fútbol

El modelado, despliegue UV y la asignación de materiales del balón de fútbol se realizaron íntegramente en la interfaz interactiva de Blender. Para recrear la estructura geométrica clásica de un balón se partió de un icosaedro truncado (compuesto por 12 caras pentagonales y 20 hexagonales) proyectado a una forma esférica de radio 0.6 ubicada en $(-5{,}5, 3, 0{,}6)$.

En el modo edición de Blender, se configuró el mapeo de coordenadas UV para cada uno de los parches. Se crearon dos materiales basados en la textura de cuero `Leather012` [2]: un material de cuero claro asignado a las caras hexagonales y una variante de cuero oscuro (con base entintada al 12 %) asignada a las caras pentagonales. Las costuras entre los paneles se definieron marcando aristas biseladas para brindar relieve visual a las uniones. Finalmente, se aplicó sombreado suave (*Shade Smooth*) para lograr una curvatura esférica continua y uniforme en la superficie del balón.

3. Experimentos

En esta sección se describen las pruebas que hicimos en cada actividad, incluidas las que fallaron, porque varias de las decisiones de la metodología salieron precisamente de esas fallas.

3.1. Experimentos de la Actividad 1: canal alfa de la hoja

3.1.1. Objetivo del experimento

Obtener una textura RGBA de la hoja a partir de una fotografía que no trae canal alfa, y comprobar que la transparencia se conserva desde Blender hasta OpenGL.

3.1.2. Pruebas fallidas y análisis de errores

Máscara elíptica. Como la hoja tiene forma casi elíptica, la primera versión del alfa fue una elipse con borde suave ajustada a mano, con centro $(371, 364)$ y semiejes 223×242. Al componer el resultado sobre magenta (Figura 5, izquierda) aparecieron dos defectos: la punta superior de la hoja es más angosta

que la elipse, así que se coló fondo arriba a la izquierda, y en la parte de abajo quedó visible la yema del pulgar.

Umbral de color. La segunda versión separó la hoja por su color y se quedó con la región conectada al centro (Figura 5, centro). La orilla quedó mucho más fiel, pero con tres defectos nuevos: la punta superior, más clara que el resto, no pasó el umbral; dos manchas del fondo del mismo naranja quedaron pegadas a los costados, y apareció un hueco abajo a la izquierda.

La versión final combina las dos ideas: el umbral de color da la orilla y la elipse, agrandada un 4 %, sirve solo como límite para cortar las manchas; el relleno por filas recupera la punta y los huecos (Figura 5, derecha).



Figura 5: Tres versiones del canal alfa compuestas sobre magenta: elipse ajustada a mano (izquierda), umbral de color con región conexa (centro) y combinación final con límite elíptico y relleno por filas (derecha).

Un falso negativo en la verificación. Al revisar la primera textura con un guion auxiliar, este reportó que el alfa valía 1 en todos los píxeles, como si la máscara no se hubiera guardado. Antes de cambiar el código que la generaba comprobamos por separado la escritura y la lectura del PNG, y las dos eran correctas: el error estaba en el guion de verificación, que imprimía el alfa de una *vista* del arreglo de NumPy después de haberlo sobrescrito con 1. La textura siempre había estado bien.

El nombre del nodo. El guion funcionó en la prueba sin interfaz, pero al ejecutarlo con Blender abierto falló al buscar el nodo con `nodes.get("Principled BSDF")`: la interfaz de Blender está en español y el nodo se llama de otra forma. Desde entonces los nodos se buscan por su tipo, `BSDF_PRINCIPLED`, que no depende del idioma.

3.1.3. Pruebas exitosas

Cargada en OpenGL, la hoja se dibuja con su contorno recortado y el pasto del fondo se ve alrededor (Figura 13), lo que confirma que el canal alfa llega intacto hasta el *shader* a través de Assimp y `stb_image`: `TextureFromFile` detecta los 4 canales del PNG y sube la imagen como `GL_RGBA`.

3.2. Experimentos de la Actividad 2: posición de la caja

3.2.1. Objetivo del experimento

Verificar que varias figuras exportadas en un mismo FBX conservan en OpenGL la posición que tienen en Blender.

3.2.2. Pruebas fallidas y análisis de errores

Al exportar la hoja y la caja juntas, la caja apareció en el centro de la escena y la hoja desapareció (Figura 6). En Blender la caja está en $x = 3$, pero el exportador guarda esa posición en la matriz del nodo y deja los vértices en coordenadas locales, alrededor del origen. Revisando `model.h` confirmamos que `processNode` recorre los nodos sin aplicar nunca `mTransformation`, así que todas las mallas se dibujan en su espacio local: la caja, de 2 unidades por lado, quedó centrada en el origen y la hoja, que está en $z = 0$, quedó dentro de ella.



Figura 6: Primera exportación de la hoja y la caja: la caja se dibuja en el origen, ignorando su posición, y oculta a la hoja.

Había dos soluciones: aplicar las transformaciones en Blender (*Ctrl+A*) o hacerlo solo en el archivo exportado. Elegimos la segunda, exportando copias temporales con la transformación ya aplicada a los vértices, porque la primera mueve el origen de cada objeto al origen del mundo y ya no se puede reacomodar la escena con comodidad.

3.2.3. Pruebas exitosas

Con la exportación corregida, la caja aparece a la derecha de la hoja y las dos se ven completas (Figura 7).



Figura 7: Hoja y caja en sus posiciones después de exportar las mallas con la transformación aplicada.

3.3. Experimentos de la Actividad 3: texturas y forma del barril

3.3.1. Objetivo del experimento

Elegir las imágenes que mejor representan un barril y comprobar que la edición proporcional produce la forma abombada sin alterar las tapas.

3.3.2. Pruebas exitosas

Comparamos tres texturas de madera y cuatro de metal (Figura 8). De las maderas descartamos `Planks012` y `Planks020` para las duelas porque son pisos de tablas cortas con juntas a tope, que en el costado del barril se verían como duelas partidas; `Planks012` sí se usó en las tapas, donde las tablas cortas no estorban. De los metales escogimos `Metal021`, acero gris azulado con puntos de óxido, porque contrasta con la madera clara; `Metal011` se reservó para la reja.

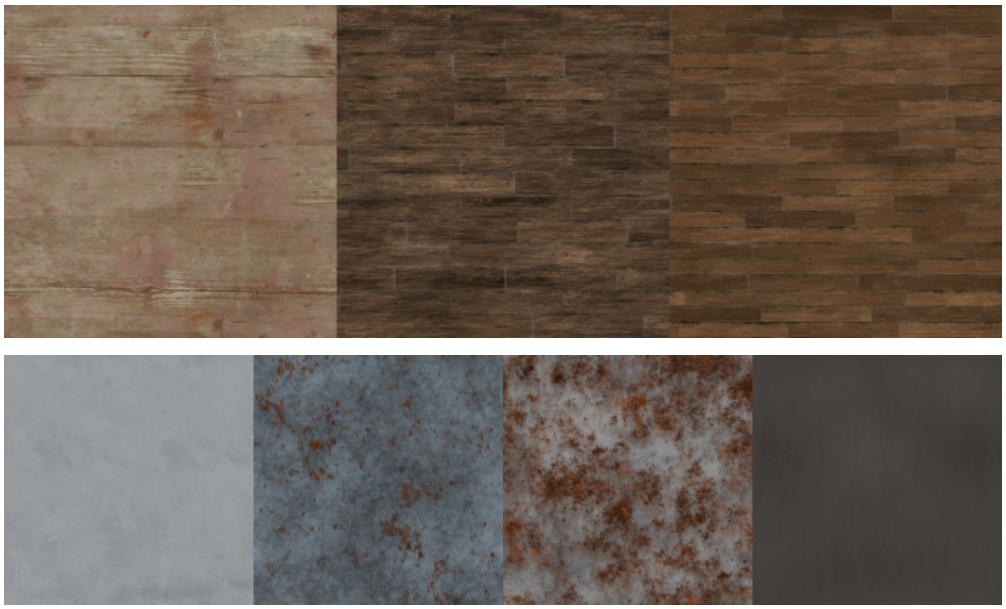


Figura 8: Texturas candidatas de ambientCG. Arriba, de izquierda a derecha: `Planks005`, `Planks012` y `Planks020`. Abajo: `Metal011`, `Metal021`, `Metal026` y `Metal036`.

Para verificar la edición proporcional medimos el radio de los anillos después de escalar: 1.0 en los anillos de las tapas y 1.18 en el central, como se buscaba. La Figura 9 compara el barril antes y después en OpenGL; los aros siguen la curva porque la subdivisión interpoló las coordenadas UV.



Figura 9: El barril en OpenGL como cilindro recto (izquierda) y después de la edición proporcional (derecha).

3.4. Experimentos de la Actividad 4: la esfera sin normales

3.4.1. Objetivo del experimento

Cargar la esfera texturizada junto con las figuras anteriores y comprobar que la textura equirrectangular se ve sin deformaciones.

3.4.2. Pruebas fallidas y análisis de errores

Cierre del programa. Al agregar la esfera al FBX, el programa mostraba la ventana en negro y se cerraba (Figura 10). El código de salida era `0xC0000005`, una violación de acceso, y el visor de eventos de Windows indicaba que ocurría dentro del propio `Proyecto06.exe`. Lo ejecutamos con el depurador de Visual Studio y la pila de llamadas lo ubicó en `Model::processMesh`, en la línea 173 de `model.h`, que lee `mesh->mNormals[i]`. Al inspeccionar las variables encontramos que Assimp entregaba la esfera (`Esfera.001`, con 8 064 vértices y 3 968 triángulos) con `mNormals` y `mTangents` en `NULL`: la esfera es la única malla con sombreado suave, y sus normales no llegaban en la forma en que Assimp las lee, mientras que las de la caja y el plano sí. El cargador del laboratorio da por hecho que toda malla trae normales y las lee sin revisar.



Figura 10: Ventana del programa al cargar la escena con la esfera, antes de cerrarse con la violación de acceso `0xC0000005`.

La corrección fue agregar `aiProcess_GenSmoothNormals` a las banderas de `ReadFile`. Esa etapa de Assimp calcula normales suaves únicamente para las mallas que no las traen, así que no altera la caja ni el plano, y como corre antes de `aiProcess_CalcTangentSpace`, las tangentes también quedan calculadas.

La caja borrada. El primer intento de crear la esfera falló dentro de Blender porque el operador `shade_smooth` no tiene contexto cuando se ejecuta desde un temporizador. Al repetir el guion, este borró el objeto llamado «Planeta» antes de crear uno nuevo, y el objeto que se llevó fue la caja: en el primer intento el guion había tomado el objeto recién creado como `bpy.context.active_object`, que dentro de un temporizador todavía apuntaba a la caja, y la había renombrado. Lo detectamos porque la exportación siguiente listó solo dos mallas. Desde entonces los guiones identifican el objeto nuevo comparando el conjunto de objetos antes y después de crearlo, y el sombreado suave se aplica con `mesh.shade_smooth()`, que no depende del contexto. La caja se volvió a generar con su guion.

3.4.3. Pruebas exitosas

Con la corrección, la Tierra se ve con los continentes en su posición y sin deformaciones cerca del ecuador ni de los polos, y América no aparece espejeada, lo que confirma que la orientación del u de la esfera UV de Blender coincide con la del mapa.

3.5. Experimentos de la Actividad 5: origen del mapa de entorno

3.5.1. Objetivo del experimento

Sustituir el fondo sin modificar el modelo del cubo de entorno ni las imágenes que usan las demás prácticas.

3.5.2. Pruebas exitosas

Al copiar `mycube.fbx` a su propia carpeta junto con una carpeta `mycube.fbm` con las seis caras de *Footballfield*, el programa cargó el nuevo fondo a la primera, y las caras quedaron en su lugar: el cielo arriba, el pasto abajo y el horizonte continuo entre las cuatro caras laterales. Esto confirma que el cubo del laboratorio fue armado con la misma convención de nombres que usa Humus (`posx` a `negz`).

3.6. Experimentos de las variaciones

3.6.1. Objetivo del experimento

Comprobar que las técnicas de las cuatro actividades se generalizan a objetos distintos y que las texturas de las variaciones (la malla de la reja y el patrón del balón) son correctas.

3.6.2. Pruebas fallidas y análisis de errores

Hexágonos desalineados. La primera textura del balón tenía unos pentágonos deformes, rodeados por hexágonos enormes (Figura 11, izquierda). Para revisarla no bastaba con la imagen equirrectangular, que deforma los polos, así que dibujamos el patrón visto de frente en proyección ortográfica. El producto punto entre los centros vecinos dio la pista: entre pentágonos valía 0.4472, el valor correcto del icosaedro, pero los centros de hexágono de la forma $(0, \pm 1/\varphi, \pm\varphi)$ no formaban el dodecaedro dual de *ese* icosaedro, sino uno girado. El dual correcto tiene los vértices en $(\pm 1/\varphi, 0, \pm\varphi)$ y sus permutaciones cíclicas; con ese cambio el patrón quedó como el de un balón real (Figura 11, derecha).

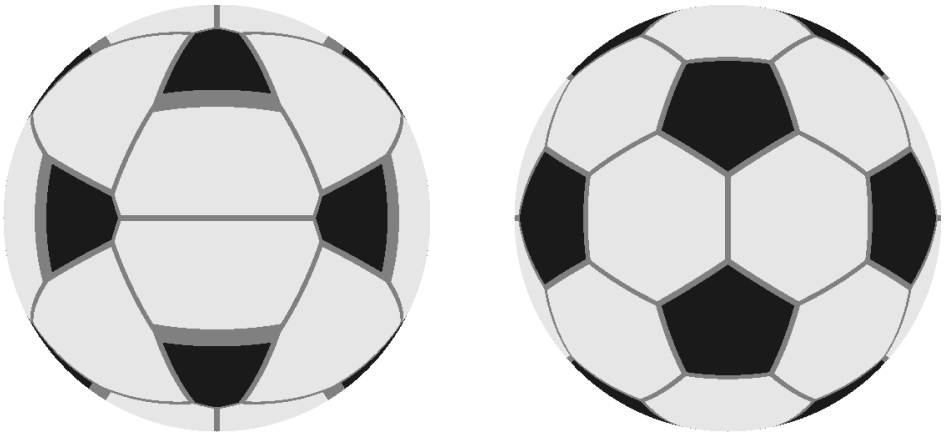


Figura 11: Patrón del balón visto de frente: con el dodecaedro girado (izquierda) y con el dodecaedro dual correcto (derecha).

Tinaco demasiado oscuro. La foto `Plastic006` es casi negra, y en la primera versión las estrías del tinaco no se distinguían. Se aclaró la base $(0{,}07 + 1{,}8\,c)$ y se cambió la modulación de las estrías por $\cos^8$, que deja franjas angostas y marcadas en lugar de una variación suave.

3.6.3. Pruebas exitosas

La reja se dibuja delante del edificio y, a través de sus rombos, se ve completa la fachada que está detrás. Esto depende del descarte de fragmentos en el *shader* (`discard` si $\alpha < 0{,}1$): el orden de dibujo de las ocho mallas dentro del modelo lo decide el FBX y no el programa, así que dibujar «primero lo opaco» no se puede garantizar, y sin el descarte los píxeles transparentes de la reja escribirían su profundidad y

ocultarían lo que se dibujara después detrás de ella. Por la misma razón, el edificio se ve a través de los huecos de la reja sin importar desde qué lado se mire.

4. Resultados

A continuación se presentan los resultados de cada actividad y de las variaciones. Todas las capturas de OpenGL corresponden al programa compilado en Visual Studio con la configuración `Debug|x64`.

4.1. Resultados de la Actividad 1: hoja en un plano

En Blender, con el sombreado de material, el plano muestra la hoja recortada y deja ver la cuadrícula del piso alrededor de su contorno (Figura 12). En OpenGL el resultado es el mismo sobre el escenario: la hoja queda tendida en el piso y el fondo se ve a través de las zonas transparentes del plano, sin un recuadro alrededor (Figura 13). La orilla, suavizada en unos dos píxeles, no muestra escalones al verse en perspectiva.

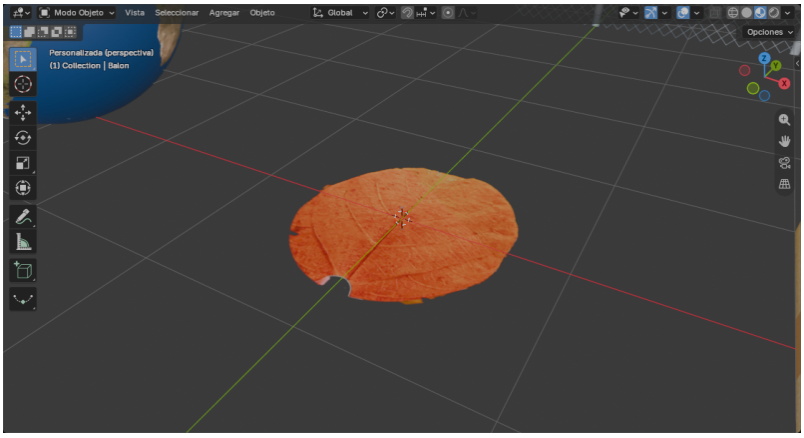


Figura 12: Plano con la textura RGBA de la hoja en la vista de Blender (sombreado de material).



Figura 13: La hoja en OpenGL sobre el mapa de entorno original: el fondo se ve alrededor de su contorno.

4.1.1. Análisis de ventajas y desventajas

- **Ventajas:** Un solo plano de 4 vértices representa un objeto de contorno irregular, que con geometría habría requerido decenas de vértices; y como el alfa se calcula con un guion, el procedimiento se puede repetir con otra fotografía cambiando solo los umbrales.
- **Desventajas:** La hoja es plana y de lado desaparece, y el canal alfa depende de que el objeto tenga un color distinto del fondo: en esta foto hizo falta un límite geométrico adicional para separarlos.

4.2. Resultados de la Actividad 2: caja de cartón

La Figura 14 muestra el atlas de la caja: la fila inferior contiene el frente, el costado derecho con la cinta que baja y la parte de atrás; la segunda fila, el costado izquierdo, la tapa con la pestaña y la cinta, y el fondo. En Blender (Figura 15) se ve que la cinta de la tapa coincide con la que baja por el costado, lo que confirma que la orientación de cada cara en el atlas es la correcta, y que el «arriba» de todos los costados apunta hacia $+Z$.

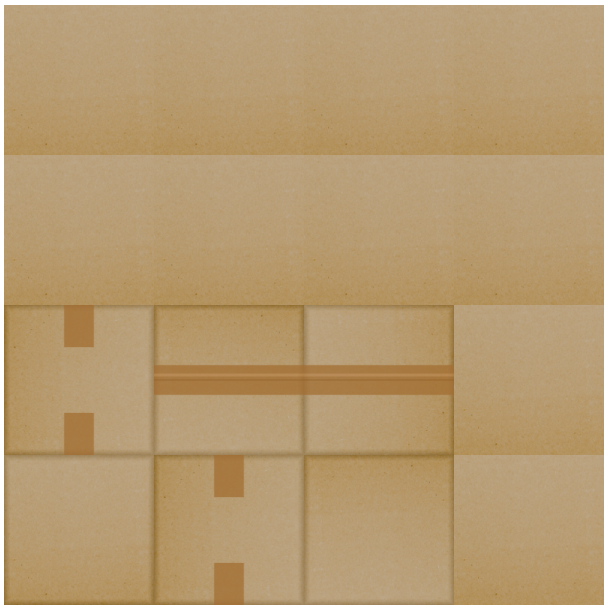


Figura 14: Atlas de la caja (2048×2048, cuadrícula de 4×4 celdas de 512 píxeles, de las que se usan seis).



Figura 15: La caja de cartón en Blender: la cinta cruza la tapa y continúa por el costado.

4.2.1. Análisis de ventajas y desventajas

- **Ventajas:** Con un atlas, el cubo usa un solo material y una sola textura, que en OpenGL es una sola consulta de imagen; además, cada cara puede llevar un diseño distinto, como la cinta que solo aparece en la tapa, el fondo y dos costados.
- **Desventajas:** Diez de las dieciséis celdas del atlas no se usan, y el margen de 4 píxeles por celda es necesario para que el filtrado no mezcle caras vecinas.

4.3. Resultados de la Actividad 3: barril

El atlas del barril (Figura 16) reúne las tres imágenes: las duelas con sus aros y remaches en la mitad superior y las dos tapas en la inferior. En Blender (Figura 17) el costado se ve continuo, sin la franja que

dejaría la costura, y la forma abombada hace que los aros superior e inferior queden en la parte angosta, como en un barril real. Desde arriba se ve la tapa de tablas oscuras con su borde de acero.



Figura 16: Atlas del barril (2048 × 2048): costado con duelas, aros y remaches arriba; tapa y fondo abajo.

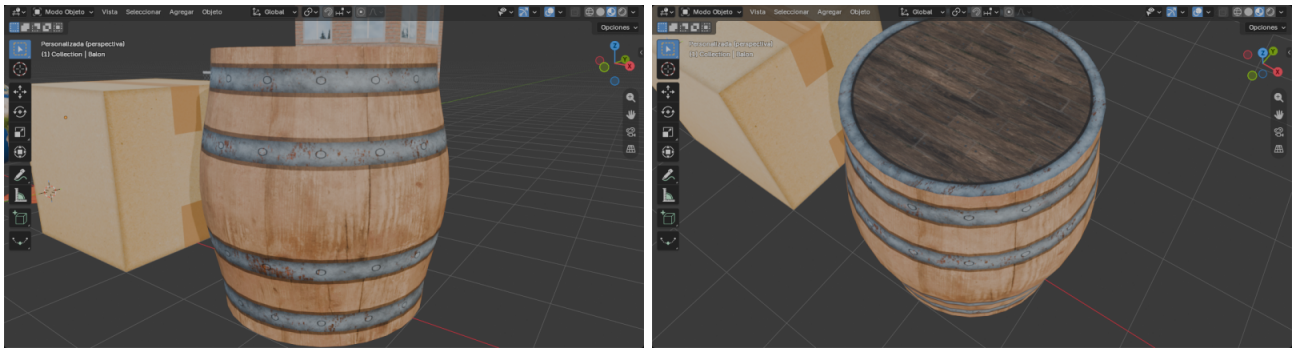


Figura 17: El barril en Blender después de la edición proporcional: costado (izquierda) y vista desde arriba con la tapa (derecha).

4.3.1. Análisis de ventajas y desventajas

- **Ventajas:** La forma se obtuvo escalando un solo anillo, y la edición proporcional repartió el cambio entre los demás de forma suave; como se hizo después de texturizar, las coordenadas UV se conservaron sin trabajo adicional.
- **Desventajas:** Al abombar el costado, la textura se estira un 18 % a lo ancho en el centro, porque las coordenadas UV no se recalcularon con la nueva circunferencia; en la madera no se nota, pero en una textura con figuras regulares sí se vería.

4.4. Resultados de la Actividad 4: planeta

La Tierra se ve correctamente en Blender y en OpenGL (Figura 18): los continentes conservan su forma y el hielo del Ártico queda arriba, sin pellizcos visibles en los polos gracias a los 32 anillos de la esfera.

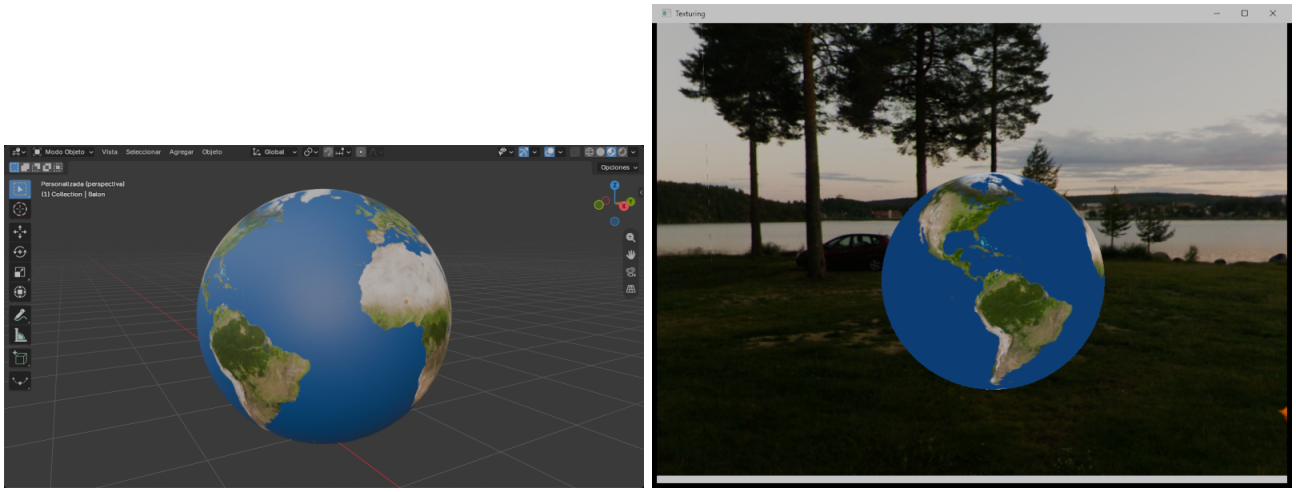


Figura 18: El planeta en Blender (izquierda) y en OpenGL (derecha).

4.4.1. Análisis de ventajas y desventajas

- **Ventajas:** La esfera UV y la proyección equirrectangular son compatibles de origen, así que cualquier mapa de planeta se aplica sin desplegar nada.
- **Desventajas:** La proyección equirrectangular desperdicia resolución cerca de los polos, donde toda una fila de la imagen se concentra en un círculo pequeño, y los triángulos de los polos convergen en un solo vértice.

4.5. Resultados de la Actividad 5: nuevo mapa de entorno

La Figura 19 compara la misma escena con el mapa de entorno original del laboratorio y con el cubemap *Footballfield*. El cambio no requirió tocar el modelo del cubo ni sus coordenadas de textura, solo las seis imágenes y una ruta en el código. Con la cancha de fondo, el pasto continúa visualmente el piso sobre el que están las figuras y la escena gana coherencia con el balón y la reja.

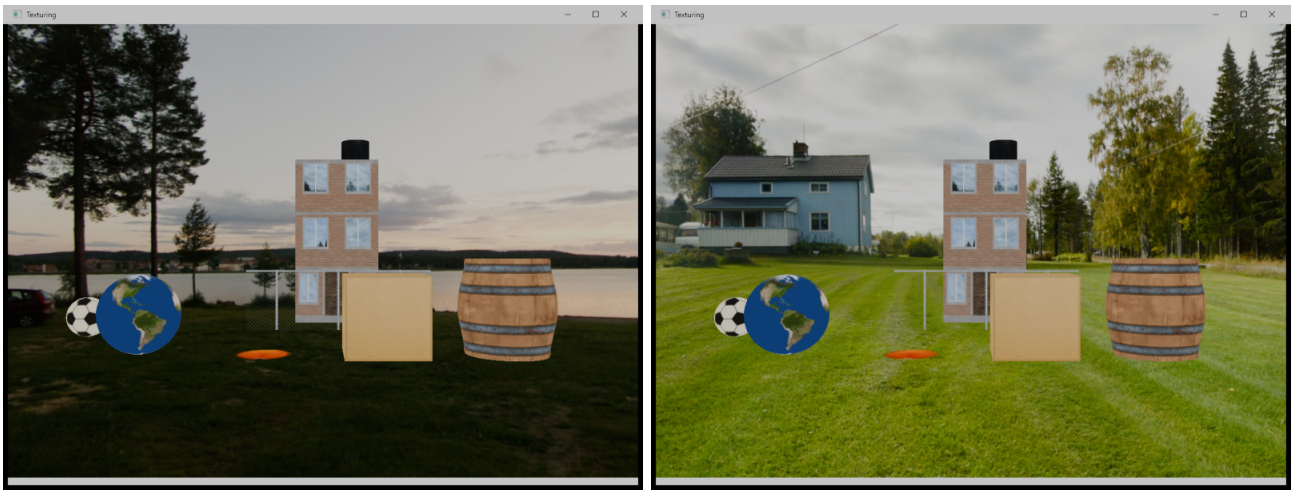


Figura 19: La escena con el mapa de entorno original (izquierda) y con el cubemap *Footballfield* de Humus (derecha).

4.5.1. Análisis de ventajas y desventajas

- **Ventajas:** Seis imágenes de 2048×2048 dan un fondo fotográfico completo a cualquier dirección de la cámara con solo doce triángulos.
- **Desventajas:** El fondo no tiene paralaje: al mover la cámara, los árboles y la casa del fondo se desplazan junto con ella, y las figuras no proyectan sombra sobre el pasto de la imagen.

4.6. Resultados de las variaciones

La Figura 20 reúne las cuatro texturas generadas. En la reja, el magenta es la parte transparente y solo quedan opacos los rombos, el poste y el tubo, que suman el 16 % de la imagen. En el atlas del edificio se distinguen las cuatro fachadas con la puerta en la planta baja de la primera, y en la textura equirrectangular del balón, los 12 pentágonos y 20 hexágonos deformados por la proyección.

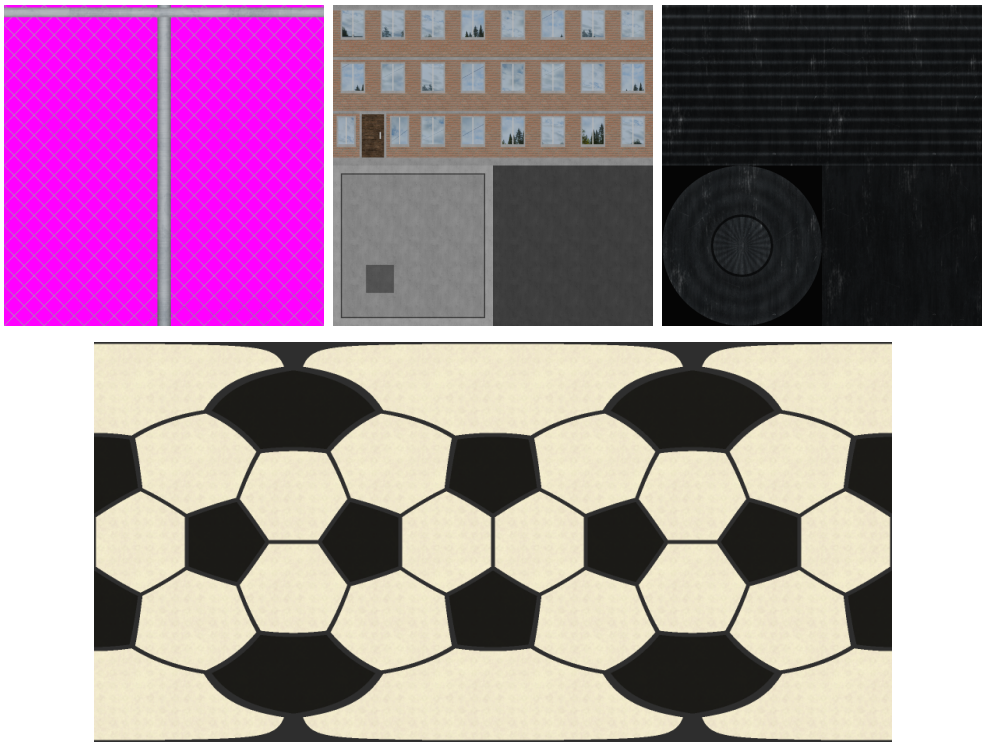


Figura 20: Texturas de las variaciones: reja sobre magenta, atlas del edificio y atlas del tinaco (arriba), y textura equirrectangular del balón (abajo).

En Blender (Figura 21) el edificio muestra sus ventanas con el reflejo del cielo, la puerta, las losas de cada piso y el tinaco sobre la azotea; detrás de la reja se ve la fachada a través de los rombos, y el balón tiene el patrón completo de pentágonos y hexágonos.

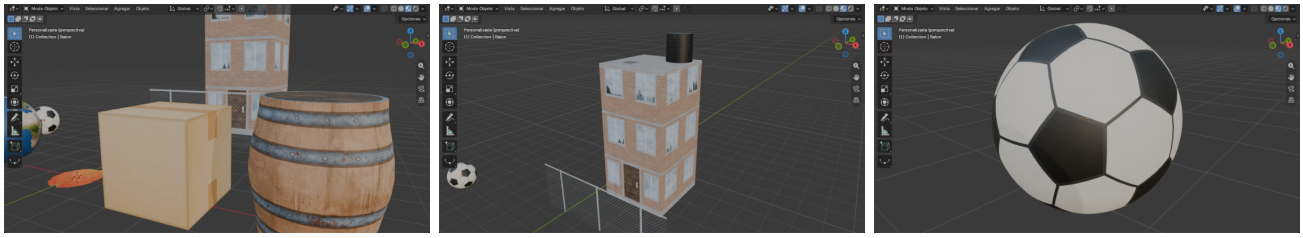


Figura 21: Variaciones en Blender: reja con el edificio detrás (izquierda), edificio con tinaco (centro) y balón de fútbol (derecha).

Finalmente, la Figura 22 muestra la escena completa. En OpenGL, la reja es transparente sin que la fachada que está detrás desaparezca, el tinaco queda sobre la azotea y el balón junto al planeta.

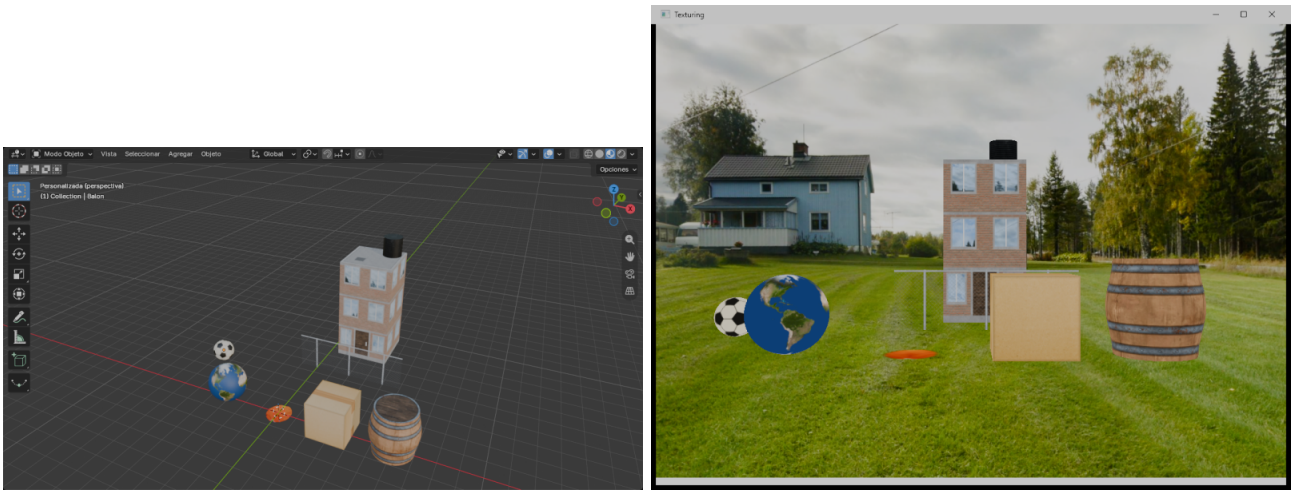


Figura 22: Las ocho figuras en Blender (izquierda) y en OpenGL con el nuevo mapa de entorno (derecha).

4.6.1. Análisis de ventajas y desventajas

- **Ventajas:** Las variaciones reutilizaron las mismas técnicas de las actividades con otros parámetros, y las texturas calculadas (la malla de la reja y el balón) no dependen de encontrar una imagen adecuada en internet y se pueden generar a cualquier resolución.
- **Desventajas:** Las ventanas del edificio son solo textura y reflejan un cielo fijo que no cambia con el punto de vista; y el tinaco, al estar en el mismo modelo que el edificio, no se puede mover por separado en OpenGL.

5. Conclusiones Individuales

#321137135: Basilio Illescas Andrés

Los objetivos de la práctica se cumplieron: las cuatro primitivas quedaron texturizadas como una hoja, una caja, un barril y un planeta, cada una con su variación, y las ocho se cargan en OpenGL sobre un mapa de entorno nuevo. Lo que me llevo es que texturizar no consiste en «poner una imagen» sobre un objeto, sino en decidir, para cada punto de la superficie, qué punto de la imagen le corresponde. Esa decisión es distinta para cada primitiva: en el plano es directa, en el cubo hay que repartir seis caras en un atlas y orientar cada una para que la cinta de la tapa coincida con la del costado, en el cilindro aparece una costura que, si no se corrige, deja una franja con toda la textura comprimida, y en la esfera la proyección equirrectangular resulta ser exactamente la que ya trae la esfera UV de Blender. Una vez visto así, el balón de fútbol fue el mismo problema al revés: en lugar de buscar una imagen para la esfera, calculamos la imagen a partir de la geometría del icosaedro truncado.

La práctica también me dejó una lección sobre la verificación misma. El primer guion con el que revisé la hoja me dijo que el canal alfa valía 1 en todos los píxeles, y estuve a punto de reescribir la máscara, que estaba bien; el error estaba en el guion que la revisaba. Algo parecido pasó con el balón: la imagen equirrectangular deforma tanto los polos que, a simple vista, no se podía saber si el patrón era correcto, y lo que reveló la falla fue dibujarlo de frente y medir el producto punto entre los centros vecinos. Por eso, para comprobar que las texturas medían potencias de dos, leímos el ancho y el alto directamente del encabezado de cada archivo en lugar de confiar en lo que reportaba Blender.

Si en la Práctica 03 me quedé con que «se ve bien» no equivale a «está bien», en la 04 con que dos cosas que se ven iguales no necesariamente lo son, y en la 05 con que algo funcione con un ejemplo no demuestra que funcione en general, aquí me queda una cuarta versión: también hay que verificar al verificador, y todo cargador da algo por sentado. El de este laboratorio daba por hecho que cada nodo del FBX estaba en el origen y que toda malla traía normales; mientras usamos sus modelos de ejemplo esas suposiciones nunca se notaron, y al traer los nuestros aparecieron como una caja en el lugar equivocado y una violación de acceso. En los dos casos el síntoma no apuntaba a la causa, y lo que la encontró fue el depurador: la pila de llamadas, la línea 173 y un apuntador en `NULL`.

#424065465: Cruz Macedo Samuel Santiago

Entrando a esta práctica pensé que texturizar era la parte sencilla del modelado: elegir una foto bonita y aplicarla. Fue al revés. La foto casi nunca venía como la necesitábamos: la de la hoja no tenía canal alfa y hubo que fabricarlo, la del cartón era solo una superficie lisa a la que le tuvimos que dibujar la pestaña y la cinta, y para el barril ninguna imagen servía sola, así que lo armamos con tres: las tablas para las duelas, otra madera para las tapas y una foto de acero oxidado para los aros. Lo que me hizo cambiar de idea fue entender que el trabajo de verdad no está en la imagen sino en las coordenadas UV, en decidir qué pedazo de la textura va en qué parte del objeto.

Lo que más me sorprendió fue la caja que apareció en el lugar equivocado. En Blender estaba a la derecha de la hoja y en OpenGL salía en el centro, tapándola. Ya en la Práctica 05 habíamos visto que `model.h` ignora las transformaciones de los nodos del FBX, pero ahí la solución fue aplicar todas las transformaciones antes de exportar; aquí preferimos exportar copias con la posición ya aplicada, para que en Blender cada objeto conservara su origen. Fue la misma limitación vista dos veces, y la segunda vez la reconocimos por el síntoma antes de buscar en el código.

De la Actividad 5 me quedo con lo mucho que cambia una escena el fondo. Con el lago original las figuras parecían flotar frente a un paisaje que no tenía nada que ver con ellas, y con la cancha de Humus el pasto del fondo continúa el piso donde están, de modo que el balón y la reja parecen estar en su lugar. También aprendí que un cubemap son solo seis fotos con una convención de nombres, y que gracias a esa convención pudimos cambiar el fondo sin tocar el modelo del cubo. Considero que los objetivos se cumplieron, y me queda claro que en el proyecto final gran parte del realismo va a depender de las texturas más que de la cantidad de polígonos.

#321258173: Medel Piña Ángel de Jesús

Cumplimos los objetivos de la práctica: texturizamos las cuatro primitivas como hoja, caja, barril y planeta, hicimos una variación de cada una y cargamos los ocho modelos en OpenGL sobre un mapa de entorno nuevo. Lo que más me costó entender fue que cada primitiva pide su propia forma de repartir la imagen. En el plano basta con estirarla. En el cubo acomodamos las seis caras en un atlas y giramos algunas para que la cinta de la tapa siguiera por el costado. En el cilindro, si no se corrige la costura, queda una franja con toda la textura apretada, y en la esfera resultó que la esfera UV de Blender ya trae la proyección equirrectangular que necesitábamos. El balón de fútbol lo resolvimos en sentido contrario: generamos la imagen a partir de la geometría del icosaedro truncado en lugar de buscar una que le quedara a la esfera.

Con el cambio del mapa de entorno y las variaciones me di cuenta de cuánto se puede aprovechar lo que ya está hecho. El cubo del laboratorio y el cubemap de Humus nombran sus seis caras igual, así que bastó con copiar el modelo a su propia carpeta junto con las imágenes nuevas y cambiar una ruta en el código, sin afectar a las otras prácticas. Con solo doce triángulos la cancha cubre todo el fondo, aunque al mover la cámara se nota que no tiene paralaje y que las figuras no proyectan sombra sobre el pasto. Las

variaciones reutilizaron las técnicas de las cuatro actividades con otros parámetros, como repetir la malla de la reja tres veces a lo ancho con `GL_REPEAT` o armar otro atlas para las fachadas del edificio. Aun así, cada una trajo su propio problema. En la reja, el orden en que se dibujan las mallas lo decide el FBX, por lo que la transparencia dependió de descartar en el *shader* los fragmentos con alfa menor a 0.1, y el primer patrón del balón salió deforme porque usamos un dodecaedro girado respecto al icosaedro.

#321150503: Pérez Olvera Alexis Abraham

Al iniciar esta práctica pensaba que texturizar era simplemente conseguir una imagen de internet y pegarla encima de un modelo 3D, pero rápidamente me di cuenta de que es un proceso mucho más técnico y analítico. Aprendí que el verdadero trabajo está en comprender cómo se mapean las coordenadas UV (u, v) en cada tipo de geometría. No es lo mismo texturizar un plano donde la proyección es directa, que trabajar con un cubo donde tuvimos que diseñar un atlas para distribuir cada cara y cuidar que elementos como las cintas de la caja coincidieran, o con un cilindro y una esfera, donde el reto principal fue entender cómo desenrollar la superficie sin que aparecieran costuras visibles o deformaciones extrañas en los polos.

Otro de los puntos fue entender cómo interactúa el canal alfa con el flujo gráfico en OpenGL. Al principio asumía que si un píxel era transparente simplemente no se dibujaba, pero al probar en el visor me di cuenta del problema del búfer de profundidad (z-buffer): si el fragmento transparente escribe en el búfer antes que los objetos de atrás, termina ocultándolos por completo. Comprender y solucionar esto mediante el descarte de fragmentos y el orden adecuado de renderizado fue uno de los aprendizajes más valiosos de la sesión.

Por último, la parte del mapa de entorno fue muy gratificante. Cambiar el fondo por el *cubemap* de la cancha de fútbol le dio un sentido de escala y pertenencia a todos los objetos de la escena con un costo prácticamente nulo de polígonos. Exportar la escena completa a FBX e integrarla en OpenGL con Assimp nos permitió ver el flujo de trabajo completo que se utiliza en la industria: preparar y empaquetar las texturas en Blender, cuidar las orientaciones y finalmente programar los *shaders* para que interpreten la información tal como la diseñamos. En general, considero que los objetivos de la práctica se cumplieron al cien por ciento; me llevo una visión mucho más sólida sobre cómo el manejo de texturas y materiales es lo que realmente dota de vida y realismo a un entorno tridimensional.

#320224159: Segura Cedeño Luisa María

En esta práctica aprendí que el texturizado va mucho más allá de sólo pegar una imagen sobre una figura 3D. Nos dimos cuenta de que es una parte fundamental del modelado, ya que le da muchísimo realismo a los objetos y nos evita tener que meter miles de polígonos extras para representar detalles como la de la madera, cintas o acabados metálicos.

Aprendimos conceptos súper importantes como el mapeo y las coordenadas UV, entendiendo que cada figura (plano, cubo, cilindro o esfera) requiere su propia forma de desplegar la imagen para que embone perfecto. Pero lo que me pareció más importante de todo el proceso fue la preparación previa de las imágenes de las texturas. Si no ajustamos bien la imagen desde el principio, asegurándonos de que sus dimensiones sean potencias de dos, recortándola correctamente o guardándola en el formato y extensión adecuados (como un PNG con canal alfa cuando se maneja transparencia), a la hora de pasar el modelo a OpenGL ocurren bastantes problemas: la textura puede no verse, los fragmentos transparentes pueden tapar lo que está atrás o la imagen puede sufrir deformaciones feas.

En general, entendí que preparar y configurar las texturas es un proceso bastante sencillo de hacer pero sumamente importante, ya que de ello depende totalmente el resultado visual y la calidad final de todo nuestro modelado 3D.

6. Enlace al Video Demostrativo

A continuación se proporciona el enlace al video demostrativo de la práctica, en el cual se expone de forma detallada el funcionamiento, compilación y compendio de resultados de cada una de las actividades realizadas:

Video Demostrativo

Enlace al funcionamiento de las actividades de la práctica:

`https://drive.google.com/file/d/1ACeDnnE29sSVgbC8uVQ0POQOxts8g1Mb/view?usp=sharing`

Referencias Bibliográficas

[1] Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M., & Hillaire, S. (2018). *Real-Time Rendering* (4.ª ed.). CRC Press.

[2] Demes, L. (s.f.). *ambientCG: Free public domain materials* [Texturas Bricks005, Concrete006, Leather012, Metal011, Metal021, Planks005, Planks012 y Plastic006, licencia CC0 1.0]. Recuperado de `https://ambientcg.com/`

[3] Anthropic. (2026). *Claude* [Modelo de lenguaje de gran escala]. Recuperado de `https://claude.ai/`

[4] Assimp Team. (s.f.). *Open Asset Import Library (Assimp) documentation*. Recuperado de `https://assimp.org/`

[5] Blender Foundation. (2026). *Blender 5.2 Manual: UV Unwrapping y Proportional Editing*. Recuperado de `https://docs.blender.org/manual/es/latest/`

[6] Blinn, J. F., & Newell, M. E. (1976). Texture and reflection in computer generated images. *Communications of the ACM, 19*(10), 542–547. `https://doi.org/10.1145/360349.360353`

[7] Humusak. (2014). *Cardboard-546867* [Fotografía, licencia CC0 1.0]. Wikimedia Commons. Recuperado de `https://commons.wikimedia.org/wiki/File:Cardboard-546867.jpg`

[8] Catmull, E. (1974). *A subdivision algorithm for computer display of curved surfaces* [Tesis doctoral]. University of Utah.

[9] Greene, N. (1986). Environment mapping and other applications of world projections. *IEEE Computer Graphics and Applications, 6*(11), 21–29. `https://doi.org/10.1109/MCG.1986.276658`

[10] Hearn, D., Baker, M. P., & Carithers, W. (2011). *Computer Graphics with OpenGL* (4.ª ed.). Pearson Prentice Hall.

[11] Ayiedfarith. (2020). *Perfectly Shape Leaf found at Mount Belumut, Kluang, Malaysia* [Fotografía, licencia CC BY-SA 4.0]. Wikimedia Commons. Recuperado de `https://commons.wikimedia.org/wiki/File:Perfectly_Shape_Leaf_found_at_Mount_Belumut,_Kluang,_Malaysia.png`

[12] Persson, E. (s.f.). *Textures: Footballfield* [Cubemap, licencia CC BY 3.0]. Humus. Recuperado de `https://www.humus.name/index.php?page=Textures`

[13] Khronos Group. (s.f.). *OpenGL 3.3 Reference Pages*. The Khronos Group Inc. Recuperado de `https://registry.khronos.org/OpenGL-Refpages/`

[14] de Vries, J. (2020). *Learn OpenGL: Learn modern OpenGL graphics programming in a step-by-step fashion*. Recuperado de `https://learnopengl.com/`

[15] Porter, T., & Duff, T. (1984). Compositing digital images. *ACM SIGGRAPH Computer Graphics, 18*(3), 253–259. `https://doi.org/10.1145/964965.808606`

[16] Solar System Scope. (s.f.). *Solar Textures: 2k Earth Day Map* [Textura, licencia CC BY 4.0]. Recuperado de `https://www.solarsystemscope.com/textures/`

[17] Teodoro-Vite, S. (2026). *Manual de prácticas del Laboratorio de Computación Gráfica e Interacción Humano Computadora: Práctica 06, Texturizado*. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

[18] Williams, L. (1983). Pyramidal parametrics. *ACM SIGGRAPH Computer Graphics, 17*(3), 1–11. `https://doi.org/10.1145/964967.801126`